

Algoritmos de Optimización en Grafos

Georgy Nuzhdin

U-tad

5 de diciembre de 2025



- 1 Introducción a la Optimización en Grafos
- 2 A* en detalle
- 3 Medidas de Centralidad en Grafos
- 4 Recocido Simulado
- 5 Búsqueda Tabú

- Búsqueda de caminos mínimos
 - Camino mínimo entre dos vértices (Dijkstra, A*)
 - TSP (Problema del viajero mercante)
 - TSP con restricciones (calles cortadas, rutas prohibidas...) y múltiples objetivos
- Min cut y Max flow (Fullkerson-Ford)
- Partición y clustering
 - Búsqueda de cliques
 - Búsqueda de influencers
- Matching, cubiertas y estructuras especiales

A* es un algoritmo de búsqueda que encuentra caminos de coste mínimo en grafos ponderados dirigidos o no dirigidos.

Combina la exactitud de Dijkstra con una heurística informada para guiar la búsqueda.

Garantía clave: si la heurística $h(n)$ es *admisible* (nunca sobreestima el coste real al objetivo) y *consistente* (monótona), A* devuelve una solución óptima y el orden de expansión es óptimo entre algoritmos con la misma heurística.

2.1 A* en detalle

- **Búsqueda acortada:** A diferencia de algoritmos como Dijkstra, que exploran todas las direcciones, A* utiliza información heurística para priorizar la exploración hacia el destino, lo que lo hace más eficiente.
- **Función de evaluación:** Calcula el coste total estimado ($f(n)$) para cada nodo, que es la suma del coste real desde el inicio ($g(n)$) y el coste estimado hasta el final ($h(n)$).
- **Listas de nodos:** Mantiene dos listas:
 - Lista abierta: Nodos que han sido descubiertos pero aún no se han explorado completamente.
 - Lista cerrada: Nodos que ya han sido explorados.
- **Algoritmo:** En cada paso, selecciona el nodo con la menor puntuación $f(n)$ de la lista abierta, lo mueve a la lista cerrada y añade sus vecinos a la lista abierta, recalculando sus costes si es necesario.
- **Optimalidad:** Si la función heurística es "admisible" (nunca sobreestima el coste real hasta el destino), el algoritmo garantiza encontrar la ruta óptima (la más corta).



2.2 Función de evaluación

Cada camino tiene 2 partes: lo que hemos recorrido y lo que queda.

Definamos para cada nodo n :

$g(n)$ = coste mínimo conocido desde el inicio hasta n (lo que hemos recorrido)

$h(n)$ = heurística: estimación del coste restante hasta el objetivo (lo que queda).

$f(n) = g(n) + h(n)$ es la longitud total del camino

Propiedades:

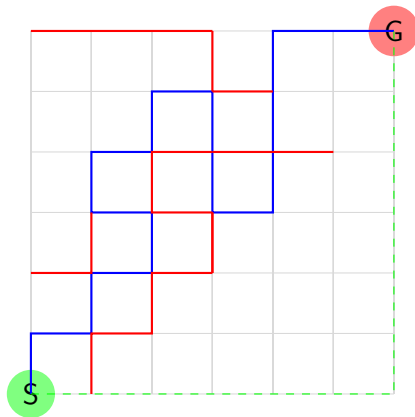
- Si $h(n) = 0$ para todo n , A^* se comporta como Dijkstra.
- **Admisible:** $h(n) \leq \text{dist}(n, \text{goal})$.
- **Consistente:** $h(n) \leq \text{dist}(n, n') + h(n')$.
- Si h es admisible y consistente, la primera vez que se saca el objetivo de la cola abierta, su g es óptimo.
- Si tenemos unos buenos predictores de distancia (cantidad de manzanas al destino en un trazado urbano, kilometraje en un desierto), la función heurística es fácil de calcular
- Si no tenemos heurística clara, usaremos Landmarks



2.3 Manhattan con calles de distintos pesos

En un trazado de calles tipo Manhattan la distancia Manhattan puede servir como heurística: aunque algunas calles pueden estar atascadas, la distancia es un buen predictor "optimista". Sin embargo, no siempre va a resultar útil.

2.4 Heurística Manhattan en un laberinto



Nota: La línea punteada verde indica la heurística Manhattan (subestima la distancia real). La línea azul es la ruta óptima.



2.5 Heurística ALT

Seleccionamos del conjunto de los vértices unos vértices que llamaremos **landmarks**.

Sea un conjunto de landmarks $L = \{\ell_1, \dots, \ell_k\}$. Precomputamos distancias:

$$\text{dist}(\ell_i, v), \quad \text{dist}(v, \ell_i)$$

para todos los nodos v .

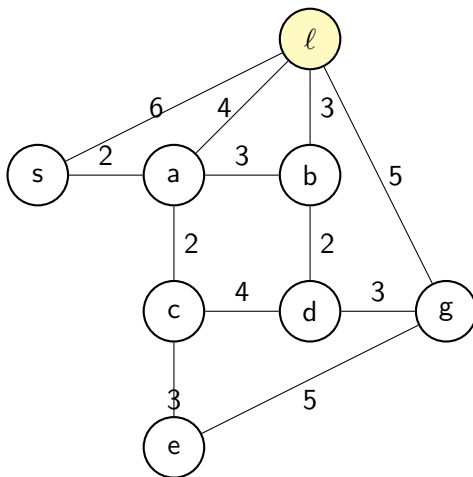
La heurística para un nodo n se define como:

$$h(n) = \max_{\ell \in L} | \text{dist}(\ell, \text{goal}) - \text{dist}(\ell, n) |.$$

- Usar el máximo mantiene admisibilidad y mejora la información.
- Por desigualdad triangular: $|d(\ell, g) - d(\ell, n)| \leq d(n, g)$.
- Por tanto, **ALT es admisible**.



2.6 Grafo de Ejemplo (para ALT)



Landmark elegido: ℓ . Vértice de salida: s . Objetivo: g .

2.7 Ejemplo: Cálculo de $h(n)$ con ALT

Precomputamos distancias desde el landmark ℓ :

$$\begin{aligned}\text{dist}(\ell, g) &= 5, \text{dist}(\ell, s) = 6, \text{dist}(\ell, a) = 4, \\ \text{dist}(\ell, b) &= 3, \text{dist}(\ell, c) = 6, \text{dist}(\ell, d) = 5, \\ &\text{dist}(\ell, e) = 9.\end{aligned}$$

Aplicamos la heurística:

$$h(n) = |\text{dist}(\ell, g) - \text{dist}(\ell, n)|$$

Ejemplos:

$$\begin{aligned}h(s) &= |5 - 6| = 1, h(a) = |5 - 4| = 1, \\ h(b) &= |5 - 3| = 2, h(c) = |5 - 6| = 1, \\ h(d) &= |5 - 5| = 0, h(e) = |9 - 6| = 3.\end{aligned}$$

El valor más informativo será usado en A^* .



2.8 A*: Ejecución paso a paso (con heurística ALT)

Costes reales (extraídos del grafo) y $h(n)$ arriba.

Nodo	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
s	0	1	1

Iteración 1: expandimos s. Hijos: a, l.

- $g(a) = 2$, $h(a) = 1$, $f(a) = 3$.
- $g(l) = 6$, $h(l) = 5$, $f(a) = 11$. Elegimos a.

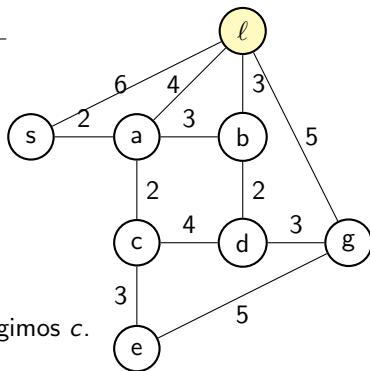
Iteración 2: expandimos a. Hijos: b, c.

- $g(b) = 2 + 3 = 5$, $h(b) = 2$, $f(b) = 7$.
- $g(c) = 2 + 2 = 4$, $h(c) = 1$, $f(c) = 5$. Elegimos c.

Iteración 3: expandimos c. Hijos: d, e.

- $g(d) = 4 + 4 = 8$, $h(d) = 0$, $f(d) = 8$.
- $g(e) = 4 + 3 = 7$, $h(e) = 3$, $f(e) = 10$. Elegimos d.

Hemos encontrado el itinerario $s \rightarrow a \rightarrow c \rightarrow d \rightarrow g$.

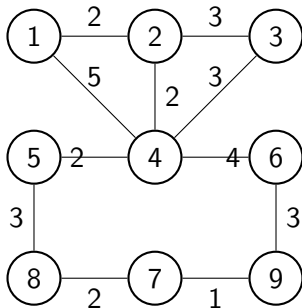


2.9 Selección de Landmarks y Ejemplo Completo

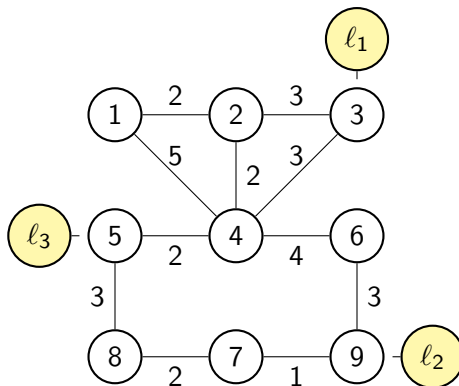
Idea: elegir landmarks alejados entre sí y situados en zonas estructuralmente distintas del grafo.

- Ejecutar Dijkstra desde cada candidato.
- Medir su "cobertura": nodos para los que produce buenas cotas triangulares.
- Elegir landmarks que maximicen la diversidad de distancias.

Elige 3 landmarks para este grafo:



2.10 Selección de Landmarks



Landmarks seleccionados: $\ell_1 = 3$, $\ell_2 = 9$, $\ell_3 = 5$.

Ejemplo: calcular $h(n)$ para encontrar la ruta óptima $1 \rightarrow 7$ usando

$$h(n) = \max_{\ell \in \{3, 5, 9\}} |d(\ell, 7) - d(\ell, n)|.$$

2.11 ¿Por qué usamos máx en ALT (no mín)?

Para cada landmark ℓ vale por desigualdad triangular:

$$|d(\ell, goal) - d(\ell, n)| \leq d(n, goal),$$

es decir $h_\ell(n) := |d(\ell, goal) - d(\ell, n)|$ es una cota inferior admisible.

Si tenemos varias cotas admisibles $h_1, h_2, \dots, h_k$, entonces

$$h_{\max}(n) := \max_i h_i(n)$$

sigue siendo admisible y normalmente es *la más informativa* de las cotas disponibles, por tanto la preferimos para guiar A*.

Nota: usar

$$h_{\min}(n) := \min_i h_i(n)$$

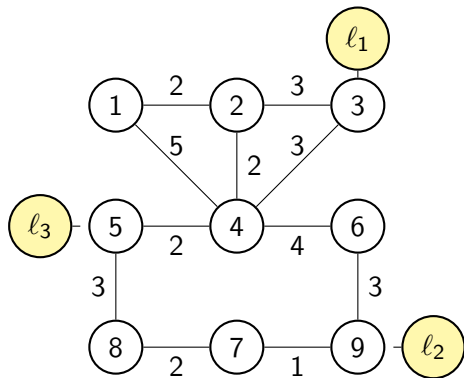
también produce una heurística admisible, pero siempre satisface $h_{\min}(n) \leq h_{\max}(n)$, por tanto es más débil (expande más nodos).

Ejemplo numérico: para el nodo 1 y landmarks $\{3, 9, 5\}$ obtuvimos

$$\{|d(3, 7) - d(3, 1)|, |d(9, 7) - d(9, 1)|, |d(5, 7) - d(5, 1)|\} = \{|10 - 5|, |1 - 11|, |5 - 11|\} = \{5, 10, 6\}$$

Entonces $h_{\max}(1) = 10$ (útil) mientras que $h_{\min}(1) = 1$ (poco útil).

2.12 Tabla: distancias desde cada landmark (3×9)

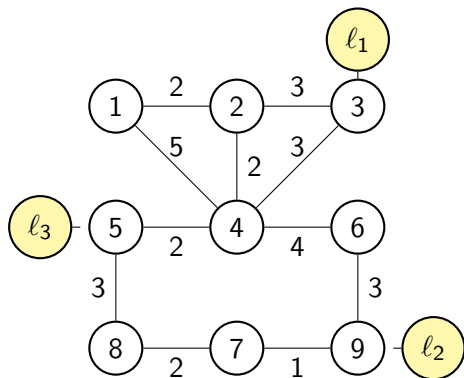


Se han elegido landmarks: $l_1 = 3$, $l_2 = 9$, $l_3 = 5$.

Nodo	1	2	3	4	5	6	7	8	9
$l_1 = 3$									
$l_2 = 9$									
$l_3 = 5$									

Haz Dijkstra desde el landmark correspondiente.

2.12 Tabla: distancias desde cada landmark (3×9)



Se han elegido landmarks: $l_1 = 3$, $l_2 = 9$, $l_3 = 5$.

Nodo	1	2	3	4	5	6	7	8	9
$l_1 = 3$	5	3	0	3	5	7	10	8	10
$l_2 = 9$	11	9	10	7	6	3	1	3	0
$l_3 = 5$	6	4	5	2	0	6	5	3	6

Cada fila es el resultado de ejecutar Dijkstra desde el landmark correspondiente.

2.13 A*: Ejecución completa (start=1, goal=7)

$$h(n) = \max_{\ell} |d(\ell, 7) - d(\ell, n)|:$$

n	1	2	3	4	5	6	7	8	9
$h(n)$	10	8	10	7	5	3	0	2	1

Paso	Exp.	g	h	f	Open
0	-	-	-	-	1(10)
1	1	0	10	10	2(10), 4(12)
2	2	2	8	10	4(11), 3(15)
3	4	4	7	11	5(11), 6(11), 3(15)
4	5	6	5	11	6(11), 8(11), 3(15)
5	6	8	3	11	8(11), 9(12), 3(15)
6	8	9	2	11	7(11), 9(12), 3(15)
7	7	11	0	11	objetivo alcanzado

Camino: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 7$, coste 11.

Nota: En el paso 4, $f(5) = f(6) = 11$. El nodo 6 se expande “innecesariamente” (su camino a 7 cuesta 4, no 3), pero no afecta al resultado: A* siempre encuentra el óptimo.

2.13.2 A*: Orden alternativo (expandiendo 6 antes que 5)

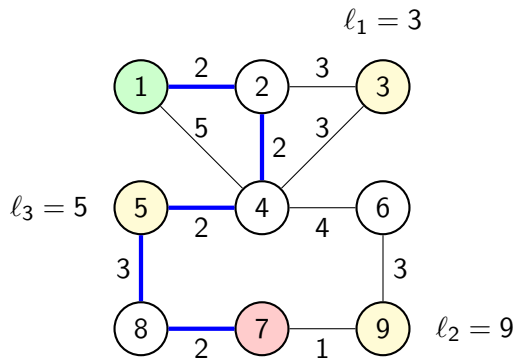
$$h(n) = \max_{\ell} |d(\ell, 7) - d(\ell, n)|: \quad \begin{array}{c|cccccccccc} n & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \hline h(n) & 10 & 8 & 10 & 7 & 5 & 3 & 0 & 2 & 1 \end{array}$$

Paso	Exp.	g	h	f	Open
0	-	-	-	-	1(10)
1	1	0	10	10	2(10), 4(12)
2	2	2	8	10	4(11), 3(15)
3	4	4	7	11	5(11), 6(11), 3(15)
4	6	8	3	11	5(11), 9(12), 3(15)
5	5	6	5	11	8(11), 9(12), 3(15)
6	8	9	2	11	7(11), 9(12), 3(15)
7	7	11	0	11	objetivo alcanzado

Camino: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 7$, coste 11.

Observación: Aunque expandimos 6 primero, el nodo 5 sigue en Open con $f = 11$. A* continúa y encuentra el camino óptimo igualmente.

2.14 Resultado: landmarks, inicio, objetivo y camino



Leyenda: amarillo = landmarks, verde = inicio, rojo = objetivo, azul = camino A*.

3.1 Betweenness Centrality (Centralidad de Intermediación)

Definición

Un nodo tiene **alta betweenness** si muchos caminos más cortos entre otros pares de nodos pasan a través de él.

Fórmula: $C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$

donde σ_{st} es el número de caminos más cortos entre s y t , y $\sigma_{st}(v)$ es el número de esos caminos que pasan por v .

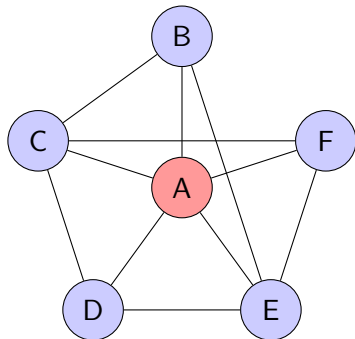
Características:

- Nodos “puente” o “cuellos de botella”
- Control del flujo de información
- Su eliminación desconecta o alarga significativamente las rutas

Ejemplos: Personas que conectan grupos sociales, routers críticos en internet.

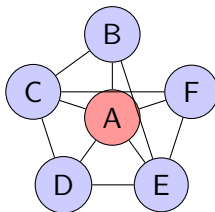


3.2 Cálculo de Centralidad de Inmediación



Par (s, t)	σ_{st}	$\sigma_{st}(A)$	$\frac{\sigma_{st}(A)}{\sigma_{st}}$

3.2 Cálculo de Centralidad de Inmediación



Par (s, t)	σ_{st}	$\sigma_{st}(A)$	$\frac{\sigma_{st}(A)}{\sigma_{st}}$
B-D	3	1	$1/3$
B-F	3	1	$1/3$
C-E	4	1	$1/4$
D-F	3	1	$1/3$

Suma Total: $1/3 + 1/3 + 1/4 + 1/3 = 5/4 = 1.25$

3.3 Closeness Centrality (Centralidad de Cercanía)

Definición

Un nodo tiene **alta closeness** si está cerca (en promedio) de todos los demás nodos del grafo.

Fórmula:
$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v,u)}$$

donde $d(v, u)$ es la distancia (longitud del camino más corto) entre v y u , y n es el número de nodos.

Características:

- Acceso rápido a toda la red
- Capacidad de difundir información eficientemente
- Independencia (no necesitan intermediarios)

Ejemplos: Almacenes centrales en redes de distribución, personas bien conectadas en organizaciones.



3.4 Betweenness vs Closeness: Diferencias Clave

Betweenness

- Mide **control/influencia** sobre el flujo entre otros
- Importancia como intermediario
- Nodos puente

Closeness

- Mide **accesibilidad** directa desde/hacia el nodo
- Eficiencia en la comunicación
- Nodos centrales

Observación importante

Un nodo puede tener:

- Alta betweenness pero baja closeness (puente entre comunidades lejanas)
- Alta closeness pero baja betweenness (centro de un grupo aislado)

3.5 Heurísticas automáticas para elegir landmarks

1. Farthest-first:

- 1 Escoge un nodo inicial (aleatorio o por centralidad).
- 2 Para $i=2..k$: selecciona el nodo cuya distancia mínima al conjunto actual de landmarks es máxima.

Ventaja: dispersión controlada.

2. Avoid / Partition-based:

- Particionar el grafo (multilevel) y escoger en cada bloque un representante (centro o nodo con mayor grado).

3. Centrality-driven:

- Escoger nodos con alta betweenness o closeness para capturar cuellos de botella.

Combinar criterios produce landmarks robustos: p.ej. farthest sobre conjuntos de candidatos centrales.



3.6 Algoritmo de Brandes para Betweenness Centrality

```
función CalcularBetweenness(G):  
    // Inicializar betweenness de todos los nodos a 0  
    para cada v en G.nodos hacer:  
        BC[v] = 0  
    para cada s en G.nodos hacer:    // Para cada nodo fuente s  
        S = pila vacía // nodos en orden creciente de distancia  
        P[w] = lista vacía para todo w // predecesores  
        s[t] = 0 para todo t; s[s] = 1 // caminos más cortos  
        d[t] = -1 para todo t; d[s] = 0 // distancias  
        Q = cola con s  
        mientras Q no vacía hacer: // BFS desde s  
            v = Q.desencolar()  
            S.apilar(v)  
            para cada vecino w de v hacer:  
                si d[w] < 0 entonces: // w no visitado  
                    Q.encolar(w); d[w] = d[v] + 1
```



3.7 Algoritmo de Brandes (continuación)

```
    si  $d[w] = d[v] + 1$  entonces: // camino más corto
         $s[w] = s[w] + s[v]$ ;
         $P[w].añadir(v)$ 
    // predecesores inmediatos de w en un camino más corto
     $\delta[v] = 0$  para todo v // dependencia de s de un nodo w
    mientras S no vacía hacer:
         $w = S.desapilar()$ 
        para cada v en  $P[w]$  hacer:
             $\delta[v] = \delta[v] + (s[v]/s[w]) \times (1 + \delta[w])$ 
        si  $w \neq s$  entonces:
             $BC[w] = BC[w] + \delta[w]$ 

// Normalizar (para grafos no dirigidos)
para cada v en G.nodos hacer:
     $BC[v] = BC[v] / 2$ 

retornar BC
```

3.8 Número de Landmarks para A*

¿Cuántos landmarks son necesarios?

No existe un número único óptimo, depende del grafo y la aplicación.

Reglas generales:

- **Mínimo:** 1-2 landmarks (mejora básica)
- **Recomendado:** $k = \sqrt{n}$ o $\log(n)$, donde n es el número de nodos
- **Típico:** 10-50 landmarks para grafos medianos (1000-10000 nodos)
- **Trade-off:** Más landmarks = mejor heurística pero mayor preprocesamiento

Ejemplo práctico

- Grafo con 1000 nodos → 15-32 landmarks
- Grafo con 10000 nodos → 32-100 landmarks
- Red de carreteras → 16-64 landmarks suele ser suficiente

4.1 Recocido Simulado en grafos

El método consiste en realizar operaciones (semi-)aleatorias aceptando empeoramiento con una probabilidad que va decreciendo con el tiempo (la "temperatura").

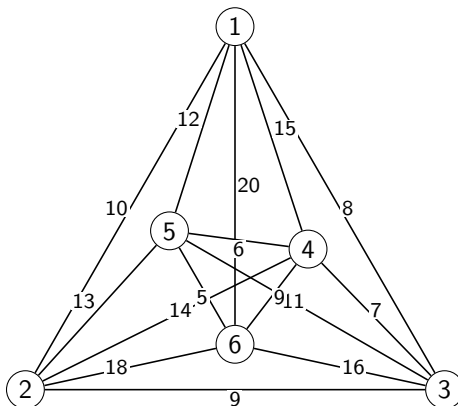
- Buscamos optimizar rutas o particiones del grafo
- En cada iteración realizamos uno de los siguiente operadores:
 - swap (intercambio de nodos)
 - 2-opt (rotura de aristas)
 - inserción de nodos
- Función objetivo: coste que queremos minimizar (normalmente, longitud de la ruta).
- Temperatura T inicial y esquema de enfriamiento.

Aceptación probabilística: $p = \exp(-\Delta C / T)$, donde ΔC es el empeoramiento después de la operación.



4.2 Recocido Simulado: TSP

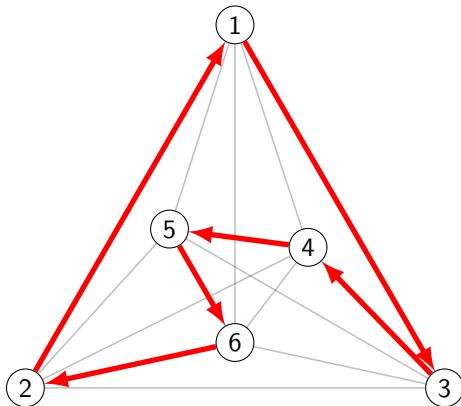
Empezamos eligiendo un recorrido hamiltoniano aleatorio:



Este grafo es completo, así que elegimos una permutación aleatoria, en el caso genérico tenemos que procurar una ruta factible



4.3 Recocido simulado: primer paso



Tour inicial: $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 2 \rightarrow 1$.

Costes aristas:

$$1 - 3 = 8, 3 - 4 = 7, 4 - 5 = 6, 5 - 6 = 5, 6 - 2 = 18, 2 - 1 = 10.$$

Coste total = $8 + 7 + 6 + 5 + 18 + 10 = 54$.

4.4 Operador SWAP (intercambio de vértices)

Idea general:

- Se eligen dos posiciones del tour $i \neq j$.
- Se intercambia el orden de esos vértices.
- Solo cambian las aristas alrededor de esos vértices.

Ejemplo:

Tour original:

$1 \rightarrow 3 \rightarrow \mathbf{4} \rightarrow 5 \rightarrow \mathbf{6} \rightarrow 2 \rightarrow 1$

swap(4,6)

Tour resultante:

$1 \rightarrow 3 \rightarrow \mathbf{6} \rightarrow 5 \rightarrow \mathbf{4} \rightarrow 2 \rightarrow 1$

Características:

- Vecindad pequeña.
- Cambio simple, buena exploración global.
- No invierte segmentos completos.



4.5 Operador 2-OPT (rotura de aristas e inversión de segmento)

Idea general:

- Elegir dos aristas del tour: (A, B) y (C, D) .
- Se cortan esas aristas.
- Se invierte el segmento de la ruta entre B y C .

Ejemplo gráfico:

Original:

$$A - [\mathbf{B} - X_1 - X_2 - \dots - X_n - \mathbf{C}] - D$$

Aplicando 2-opt:

$$A - [\mathbf{C} - X_n - X_{n-1} - \dots - X_2 - X_1 - \mathbf{B}] - D$$

Características:

- Suele usarse para eliminar cruces de aristas o aristas demasiado largas.
- Mejora muy potente para TSP.
- Swap es un caso particular pequeño de 2-opt.



4.6 Criterio general para elegir vecinos (swap / 2-opt)

Durante el recocido simulado:

1. Temperatura ALTA → Exploración

- Elegir vecinos **al azar**.
- Swap aleatorio.
- 2-opt aleatorio.

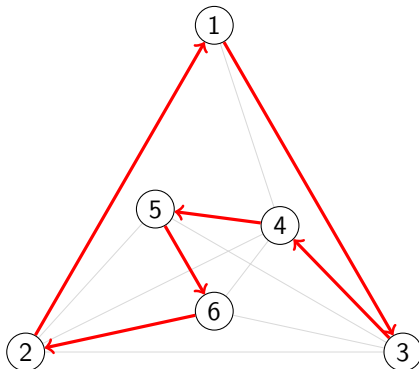
2. Temperatura MEDIA → Exploración guiada

- Priorizar aristas largas.
- Evitar swaps triviales (vértices consecutivos).

3. Temperatura BAJA → Explotación

- Aplicar 2-opt dirigido: romper aristas malas.
- Búsqueda más local y afinada.

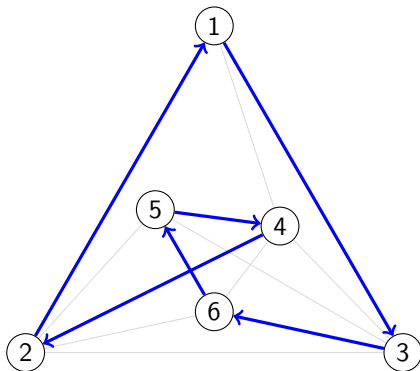
4.7 Aplicación al grafo de 6 vértices (swap y 2-opt)



Tour inicial:

$1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 2 \rightarrow 1$

4.7 Aplicación al grafo de 6 vértices (swap y 2-opt)

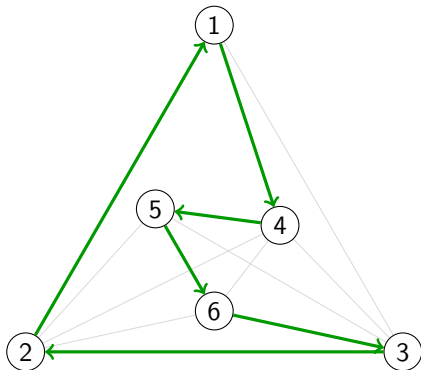


Paso 1: SWAP(4,6)

Nuevo tour: $1 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$

Está peor. Lo aceptamos con cierta probabilidad.

4.7 Aplicación al grafo de 6 vértices (swap y 2-opt)



Paso 2: 2-opt sobre aristas (1,3) y (4,2)

$1 \rightarrow 3 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ pasa a

$1 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 1$

Tour mejorado.

4.8 Recocido simulado — decisión de aceptación

Recocido simulado — decisión de aceptación

Sea $C_{current} = 54$ y $C_{new} = 59$.

$\Delta = C_{new} - C_{current} = 5$ (peor solución).

Probabilidad de aceptación (si $\Delta > 0$):

$$P = \exp\left(-\frac{\Delta}{T}\right).$$

Ejemplo numérico: $T = 10 \Rightarrow P = \exp(-5/10) = \exp(-0,5) \approx 0,6065$.

Interpretación: con $T = 10$ hay 60.6 % de aceptar este vecino peor — favorece exploración.



5.1 Búsqueda tabú

¿Qué problemas tiene cualquier heurística en grafos, como, por ejemplo, el algoritmo anterior?

5.1 Búsqueda tabú

¿Qué problemas tiene cualquier heurística en grafos, como, por ejemplo, el algoritmo anterior?

¡Fácilmente podemos entrar en bucle repitiendo los mismos movimientos!

La **búsqueda tabú (Tabu Search)** es una metaheurística de optimización que mejora la búsqueda local evitando caer en óptimos locales. Utiliza una memoria adaptativa (la lista tabú) que prohíbe temporalmente movimientos recientes para forzar la exploración de nuevas regiones del espacio de búsqueda.

Se usa típicamente en problemas de caminos cortos (TSP y otros), problemas de coloración de grafos, asignación de recursos, scheduling, y diseño de redes.



5.2 Búsqueda tabú: estructura

Componentes principales:

- **Lista Tabú:** Memoria de corto plazo que almacena movimientos recientes prohibidos durante k iteraciones
- **Criterio de aspiración:** Permite aceptar soluciones tabú si son sustancialmente mejores que la mejor conocida
- **Vecindario:** Conjunto de soluciones alcanzables desde la solución actual. Normalmente se aceptan los operadores swap, 2-opt, inserción.
- **Función objetivo:** Evalúa la calidad de cada solución

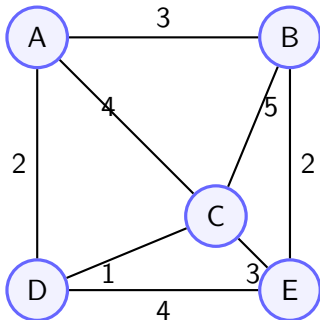
El algoritmo comienza con una solución inicial arbitraria y en cada iteración explora el vecindario, seleccionando el mejor movimiento que no esté en la lista tabú (o que cumpla el criterio de aspiración). El movimiento realizado se añade a la lista tabú por un número determinado de iteraciones (tenure).



5.3.1 Ejemplo 1: TSP

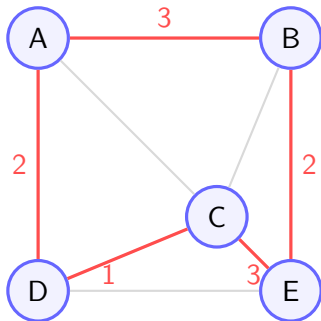
Consideremos un grafo con 5 ciudades donde queremos encontrar el ciclo hamiltoniano de menor coste.

Grafo del TSP



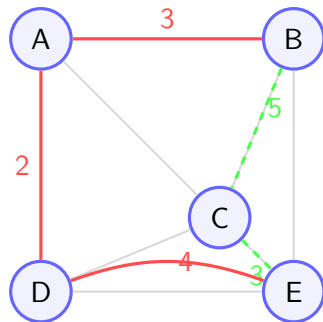
5.3.2 Ejemplo 1: TSP

Solución Inicial



Coste = 11

Iteración 1: Intercambio $(B,E,C) \leftrightarrow (B,C,E)$



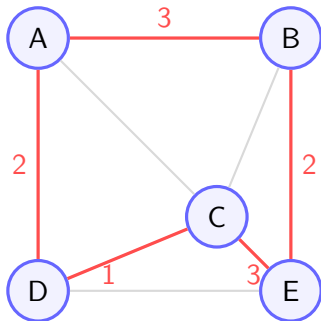
Coste = 17 (peor, pero exploramos)

Lista Tabú: $[(E,C),(B,E)]$



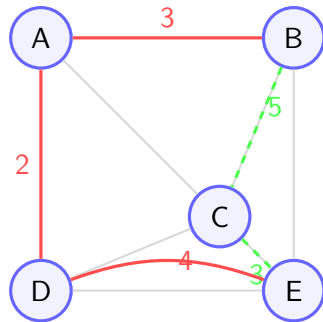
5.3.2 Ejemplo 1: TSP

Solución Inicial



Coste = 11

Iteración 1: Intercambio $(B,E,C) \leftrightarrow (B,C,E)$



Coste = 17 (peor, pero exploramos)

Lista Tabú: $[(E,C),(B,E)]$



5.4 Parámetros Tabú

- **Tamaño de la lista tabú** (tenure): Controla cuántas iteraciones un movimiento permanece prohibido. Valores típicos: 7-20 iteraciones. Un tenure corto permite más exploración pero puede ciclar, mientras que uno largo puede ser demasiado restrictivo.
- **Criterio de aspiración**: Permite aceptar movimientos tabú si producen una solución mejor que cualquier otra encontrada previamente, evitando bloquear movimientos prometedores. Podemos permitir movimientos de la lista tabú si ofrecen una mejora significativa (un 10 %, por ejemplo).
- **Estrategia de vecindario**: Define qué movimientos son posibles. Para TSP se usa comúnmente 2-opt (intercambiar dos aristas), para coloración se cambia el color de un vértice.



Aún mucho por explorar

- Problemas de persecución
- Problemas de rendez-vous
- Búsqueda de estructuras (matching, clusters, cliques, influencers...)

Algoritmos de Optimización en Grafos

Georgy Nuzhdin

U-tad

17 de diciembre de 2025



1 Introducción a la Optimización en Grafos

- Búsqueda de caminos mínimos
 - Camino mínimo entre dos vértices (Dijkstra, A*)
 - TSP (Problema del viajero mercante)
 - TSP con restricciones (calles cortadas, rutas prohibidas...) y múltiples objetivos
- Min cut y Max flow (Fullkerson-Ford)
- Partición y clustering
 - Búsqueda de cliques
 - Búsqueda de influencers
- Matching, cubiertas y estructuras especiales